

Contents

EEGdiff_V2 — Project Log	1
1. Idea in one line	1
2. The model / embedding	1
3. Datasets (patient/file-wise disjoint splits)	1
4. What we have so far (inventory)	2
5. Results so far	2
5.1 THUSZ seizure	2
5.2 TUAB abnormal (higher numbers — dataset is balanced)	3
5.3 Benchmarking vs literature (prevalence-matched)	3
5.4 Seizure-subtype classification (patient-wise LOFO CV)	4
5.5 External-dataset generalization (Bonn, EEGMMIDB; BCI IV-2a tried & dropped)	5
5.6 Siena and TUEV — adding a +RL finetuning variant	6
5.7 TUEV revisited — why was macro-F1 stuck at 0.32, and merging to 4 classes	7
5.8 sleep_edfx — a corpus-wide silent-corruption bug, found only by checking signal content	8
6. Key files	9
7. Job history	10
8. Open items / next steps	11

EEGdiff_V2 — Project Log

Last updated: 2026-07-08 (+ TUEV 4-class merge, sleep_edfx full pipeline + results) **Maintainer:** abduhl
Root: /home/abduhl/scratch/EEGdiff_V2

1. Idea in one line

Take a **diffusion model pretrained on EEG** (generative denoising objective) and reuse its encoder as a **frozen / partly-frozen feature extractor (“embedding model”)** for downstream EEG classification (seizure detection on THUSZ, abnormal-EEG detection on TUAB).

2. The model / embedding

- **Backbone:** DeepEnhancedEEGDiffusionModel (v2 “large”), **40,679,574** params.
- **Pretrained checkpoint:** training_diffusion_v2/best_EEGDIFF_V2.pth
- **ARCH:** in_channels=22, model_channels=64, channel_multipliers=[1,2,4,8], num_res_blocks=3, time_emb_dim=768, dropout=0.1, attention_heads=8
- **Embedding definition:** probe the encoder at 5 diffusion timesteps [50,250,500,750,950], adaptive-avg-pool each encoder level, concatenate -> $64 * (1+2+4+8) * 5 = 4800$ -dim embedding vector.
- **Head:** 2-layer MLP classifier on the 4800-d embedding.
- **Optional RL layer:** ReinforcedDecisionLayer (REINFORCE / Bernoulli logit adjustment). **Used only in downstream finetuning, never in pretraining.** A no-RL variant exists for the ablation.

3. Datasets (patient/file-wise disjoint splits)

Dataset	Task	Train	Val	Test/Eval	Balance
THUSZ	seizure vs non-seizure	1,858,924 (seiz 112,290)	98,000	228,667	imbalanced (~6% seizure), class weights n=0.532 / s=8.277
TUAB	normal vs abnormal	699,414	36,000	72,291	~balanced (n=0.512 / abn=0.512)

4. What we have so far (inventory)

#	Experiment	RL	Backbone	Status	Checkpoint / results location
1	THUSZ frozen	+RL	frozen	done (epoch 50)	seizure_clf_20260629_084706
2	THUSZ frozen — EVAL	+RL	frozen	done	TUHSZ_Res/run_20260702_0425
3	THUSZ unfrozen (last_two_levels)	+RL	finetune 10.6M	done (best ep22; stopped ep33, overfitting)	model Binary_finetune_unfrozen/ru
3b	THUSZ unfrozen — EVAL	+RL	finetune	done	Binary_finetune_unfrozen/ru
4	THUSZ frozen	no-RL	frozen	running (job 64564772)	Binary_finetune_norl/run_no
5	THUSZ finetune	no-RL	finetune	scripts ready, not launched	Binary_finetune_norl/
6	TUAB head_only	+RL	frozen	done (model dir since removed)	log logs/TUAB_PW_64031825.out
7	TUAB unfreeze_all	+RL	finetune 14.3M	deleted (overfit, per decision)	removed
8	TUAB unfreeze_1	+RL	finetune 7.9M	incomplete (timed out, no test)	log logs/TUAB_PW_64031825.out
9	TUAB unfreeze_2 / unfreeze_3	+RL	finetune	running (ep10, job 64561392)	TUAB_Res/run_<ts>/ (on finish)

5. Results so far

5.1 THUSZ seizure

Model	Split	ROC-AUC	PR-AUC	Seizure F1
Frozen +RL (ep50)	val	0.877	0.379	0.388 @0.5
Frozen +RL (ep50)	EVAL (unseen)	0.836	0.369	0.355 @0.5 ; 0.426 @thr 0.717
Unfrozen +RL (ep22 best)	val	0.945	0.768	0.704 @0.5 ; 0.751 @thr 0.97
Unfrozen +RL (ep22 best)	EVAL (unseen)	0.841	0.484	0.473 @0.5 ; 0.496 @thr 0.97
No-RL frozen	—	running (job 64564772)		
No-RL finetune	—	not launched		

Notes: - Best single result so far: **unfrozen +RL, EVAL ROC 0.841 / PR 0.484 / seizure F1 0.496** — clearly beats frozen (PR 0.369, F1 0.426). - The unfrozen run **overfit** if left running (train F1 -> 0.97, val loss rising 0.13 -> 0.47), so it was stopped at ep33; the saved model is the ep22 best (val ROC 0.945). Frozen remains the clean “embedding as-is” number. - For imbalanced seizure detection report **seizure F1 + PR-AUC**, not weighted-avg F1 (weighted F1 ~0.94 is dominated by the easy Normal class and is misleading).

5.2 TUAB abnormal (higher numbers — dataset is balanced)

Model	Split	ROC-AUC	PR-AUC	F1	Notes
head_only (frozen, +RL)	test	0.776	0.756	0.69	completed baseline
unfreeze_all (+RL)	test	0.842	0.821	0.74	deleted; overfit, early-stopped ep7
unfreeze_1 (+RL)	val	0.967	0.972	0.90	no test (job timed out)
unfreeze_2 / 3 (+RL)	val	~0.89	~0.90	~0.80	running (ep10), test pending

5.3 Benchmarking vs literature (prevalence-matched)

External comparison uses the **BioSerenity-E1** benchmark (arXiv 2503.10362) and its CSV of published TUAB / TUH-Seizure results. Key methodology finding: those papers report on a **class-balanced** TUH-Seizure test (~29% seizure), built by keeping seizure-containing records and **undersampling background** — NOT the natural prevalence. Our THUSZ eval is natural (~6.7% seizure), which deflates AUPRC.

So we re-scored our **saved probabilities** (`scores.npz`) after subsampling non-seizure eval windows to a target prevalence (no GPU / no retrain, 200 seeds). Script: `finetuning/benchmark_prevalence.py`; results: `Benchmarking/prevalence_benchmark.{txt,json}`.

THUSZ seizure — AUPRC vs seizure prevalence (same models, resampled test):

Model	@6.7% (natural)	@29% (matches ref)	@49% (balanced)	AUROC (all prev.)
Frozen +RL	0.369	0.712	0.838	0.836
Unfrozen +RL	0.484	0.762	0.862	0.841

Model	@6.7% (natural)	@29% (matches ref)	@49% (balanced)	AUROC (all prev.)
<i>BioSerenity-E1</i> (ref, @29%)	—	<i>AUPRC 0.83</i>	—	<i>0.926</i>

Takeaways: - The AUPRC “gap” was **mostly a prevalence artifact**: unfrozen AUPRC 0.484 -> **0.762** just by matching the 29% test construction (model unchanged). That’s near BioSerenity’s 0.83 and above several table baselines (EEGFormer 0.57, BrainBERT 0.54, FC3Net 0.69). - **AUROC is the real remaining gap** (prevalence-invariant): ours ~0.84 vs 0.926 (~0.085). Partly attributable to their **16 s windows vs our 5 s**. - At threshold 0.5, our sensitivity 0.57-0.68 vs their 0.909 (they run at higher recall); balanced accuracy ~0.76 vs 0.83.

Caveats: this fixes only the **prevalence** confound. Window length (16 s vs 5 s), sampling (128 vs 256 Hz), channels (16 vs 22), the ≥ 3 s-in-window seizure-label rule, and official-split/leakage checks are **still unmatched**. Quote **AUROC (~0.84 vs 0.926)** as the honest current gap. Full apples-to-apples needs reprocessing to 16 s / 128 Hz / 16 ch + retrain.

TUAB / TUH-Seizure comparison context (AUROC): EEG foundation models lead (CBraMod 0.923, LaBraM-Huge 0.916, BioSerenity 0.905 on TUAB; BioSerenity 0.926 on TUH-Seizure). Our current numbers sit at the **classic-CNN tier** (EEGNet ~0.84), below the foundation-model tier.

5.4 Seizure-subtype classification (patient-wise LOFO CV)

Third task: multi-class seizure-subtype classification with **strict patient-wise 5-fold CV (Leave-One-Fold-Out)** — no leakage. Everything in PW_Seizure_Subtypes/.

- Data: `.../Subtypes_L00/{train,dev,eval}`, (N,22,1280) + per-sample subtype label — patient id. 133,651 samples, **279 patients** (already disjoint across splits).
- **Classes: top-4 [1,2,7,5]** (~97% of data). Dropped subtypes 8/3/4/6 — each has ≤ 11 patients (8 has 2, 6 has 1), so they are undefined/unstable under patient-wise CV. (The old segment-wise “0.95 F1” was almost certainly leaky — same patient in train+test.)
- Folds: 279 -> after top-4 filter **264 patients** -> 5 disjoint folds (~53/fold), stratified by each patient’s rarest present subtype so sparse class 5 spreads evenly.
- Backbone: **EEGdiff_V2 frozen**; embeddings (4800-d) precomputed once, cached. Scripts: `extract_embeddings.py`, `cv_subtypes.py`, `finetune_cv_subtypes.py`.

Frozen (head-only) baseline — pooled out-of-fold (job 64570268, done):

Subtype	Precision	Recall	F1	Support
1	0.670	0.631	0.650	68,369
2	0.494	0.431	0.461	43,399
7	0.267	0.314	0.289	16,761
5	0.076	0.293	0.121	2,276

- Pooled **macro-F1 0.380**, accuracy 0.519, balanced-acc 0.417, weighted-F1 0.532.
- CV macro-F1 **0.363 +/- 0.020** (5 folds). Honest, no-leakage; far below the old leaky 0.95. Common subtypes (1,2) separate OK; rare 5/7 are weak.
- Result: PW_Seizure_Subtypes/cv_runs/run_20260702_094040/.

Finetuned + RL (job 64571047, done): unfreeze `last_two_levels` + multi-class RL (Categorical policy, macro-F1 reward) + weighted CE, per fold, early-stop on val macro-F1. Same folds. Result: **worse than frozen** — pooled macro-F1 **0.364** (vs frozen 0.380), and rare subtype 5’s F1 **collapsed 0.121 -> 0.057**. Finetuning overfit given how little data subtype 5 has; frozen remains the best subtype model. (Heavy per-fold checkpoints deleted after logging; numbers kept in `logs/SubtypesFT_64571047.out.`)

5.5 External-dataset generalization (Bonn, EEGMMIDB; BCI IV-2a tried & dropped)

Tests whether the THUSZ-pretrained embedding transfers to EEG datasets/tasks it was never trained on. Two evaluation modes throughout: - **zero-shot**: reuse the pretrained THUSZ seizure classifier as-is, no training at all. - **trained-head / finetuned**: frozen (or partially unfrozen) backbone + a **freshly trained head** on the target dataset’s own train/test split. **No RL** in any of these (plain weighted CE), by design, to isolate the unfreeze-amount effect.

Scripts: `eval_processed_binary.py` (zero-shot binary), `eval_processed_multiclass.py` (frozen/trained-head, N-class, supports `--per-window-norm / --classes / --pooled-random-split`), `finetune_processed_multiclass.py` (backbone finetune, `--unfreeze {none,last_level,last_two_levels,encoder_all,all}`). Data prep: `dataset_tools/prepare_external_eeg.py` (fixed 3 bugs along the way: macOS junk files and case-sensitive .TXT breaking Bonn, PSG/Hypnogram pairing never matching for sleep_edfx, and — most importantly — **wrong channel order and wrong sample rate** vs. the model’s actual training input; fixed to the verified TUAB channel order and 256 Hz / 1280-sample windows).

Bonn (seizure vs normal, single-channel data placed in 1 of 22 target channels, other 21 zero-filled):

Mode	ROC-AUC	PR-AUC	Accuracy
Zero-shot (THUSZ classifier, no training)	0.716	0.647	0.846
Trained head (frozen backbone + fresh head)	0.993	0.974	0.930

Embeddings transfer very well to a seizure-like task once a task-specific head is trained — despite 21/22 channels being zero. Results: `Benchmarking/bonn_binary/`, `Benchmarking/bonn_trainedhead_20260706_080252/`.

BCI IV-2a (4-class motor imagery) was tried, but its high-density fronto-central montage only maps **5/22** channels into our standard 10-20 frame (17 dropped) — near-chance result (~29%, chance=25%) was inconclusive/confounded by channel loss, not a real transfer test. **Deleted** (raw + processed + all results) rather than fixed, per decision.

EEGMMIDB (motor movement/imagery; full **22/22** channel coverage, verified — NOT a channel-loss case):

Experiment	Classes	Accuracy	Macro-F1	Notes
Frozen, no finetune	9-way	17.4%	0.111	near/below chance (rest=48% of test, only 24% recalled) worse than frozen — optimization collapse, not lack of signal
Finetune <code>last_two_levels</code> , LR-mult 0.1	9-way	7.0%	0.070	
Frozen + per-window norm	2-way (left/right fist, executed)	53.2%	0.521	ROC 0.586; chance=50%
Finetune <code>last_level</code> , LR-mult 0.01 (gentle)	2-way	57.2%	0.572	gentler LR fixed the earlier instability
Frozen, pooled-random split (diagnostic)	2-way	56.9%	0.568	ROC 0.579 — nearly identical to subject-wise split

Experiment	Classes	Accuracy	Macro-F1	Notes
Finetune all (full), LR-mult 0.01	2-way	61.5%	0.615	best result; monotonic gain with more unfreezing

Diagnosis and takeaways: - The 9-class collapse was **not** “no signal” — it was **too-aggressive optimization** (large unfrozen block + high backbone LR) on a hard, imbalanced 9-way problem, compounded by using THUSZ’s global normalization stats on a very different dataset. - **Per-window normalization** (z-score each window against its own per-channel mean/std, not THUSZ’s fixed stats) was the single highest-leverage fix. - The **pooled-random-split diagnostic** (mixes subjects between train/test, which should help a lot if cross-subject transfer were the bottleneck) barely changed the result (56.9% vs 53.2%) — ruling out cross-subject generalization as the primary limiter. - **More unfreezing monotonically helped once the LR was gentle enough**: frozen (53.2%) < last_level (57.2%) < full all (61.5%). No sign of a hard ceiling yet. - Honest read: the embedding carries **real but weak** motor-imagery-laterality signal (chance=50%, reached 61.5%) — meaningfully above chance, but far below published dedicated motor-imagery methods (70-90%+). Consistent with a backbone pretrained for seizure/abnormality detection, not sensorimotor rhythms.

Results: Benchmarking/eegmmidb_trainedhead_20260707_044455/, eegmmidb_finetuned_last_two_levels_20260707_044455/, eegmmidb_2class_subjectwise_trainedhead_20260707_052301/, eegmmidb_2class_subjectwise_finetuned_last_level_20260707_052301/, eegmmidb_2class_pooled_diagnostic_trainedhead_20260707_052448/, eegmmidb_2class_subjectwise_finetuned_all_levels_20260707_052448/

5.6 Siena and TUEV — adding a +RL finetuning variant

Follow-up to 5.5: after EEGMMIDB showed finetuning helps *if* the LR is gentle enough, we tested two more datasets AND added a **full-finetune + RL** variant (finetune_processed_rl.py — Categorical-policy REINFORCE, macro-F1 reward, same design as the subtype classifier’s RL layer, generalizes to any class count) to see whether RL helps here the way it helped THUSZ.

Siena (seizure vs normal; TUAB-style processing already done correctly by someone else — channel order/256 Hz/1280-sample all matched our spec out of the box, only fix needed was using **Siena’s own** normalization stats, since its raw signal is in volts, ~1e6x smaller scale than THUSZ-V2’s convention). Extremely imbalanced natural test prevalence: **0.70%** seizure (132/18,729) — harder than THUSZ’s 6.7% or Bonn’s curated 20%. Conversion: dataset_tools/convert_siena_npz.py (patient-wise: 11 train / 3 test patients).

Mode	Macro-F1	Seizure Precision	Seizure Recall	Seizure F1	ROC-AUC	False positives
Zero-shot (THUSZ classifier)	—	0.007	0.754	~0.014	0.626	68,179
Trained head (frozen, no RL)	0.528	0.046	0.303	0.080	0.734	825
Full finetune (all) + RL	0.638	0.366	0.227	0.280	not computed	52

RL training was clean (val macro-F1 climbed 0.51->0.83 over 40 epochs, no collapse). **Precision jumped 8x** and FPs dropped from 825 to 52, at a small recall cost — a much more clinically usable operating point. ROC-AUC not computed for the RL variant (argmax-only reporting); could add if needed.

TUEV (6-class event corpus: spsw/gped/pled/eyem/artf/bckg). **Never processed before** — built

dataset_tools/prepare_tuev.py from scratch, modeled on prepare_tuab.py’s patient-wise approach (TUEV’s own train/eval dirs are already patient-disjoint, reused directly). Two real bugs found and fixed during dev: - Windowing: TUEV’s .rec annotations are **short (~1s) discrete event markers**, not continuous session-long labels. A fixed 0/5/10s window grid (our other datasets’ approach) rarely aligns with them – first attempt threw away 118/120 windows in a single-file test. Fixed with **event-driven windowing**: extract a window centered on each merged event span instead of tiling the whole recording (merge_events + extract_event_windows). - Output folders used eval-<class>/ (TUEV’s own naming) but eval_processed_multiclass.py expects test-<class>/ – fixed by renaming.

Result: **2,621 windows** total (359 train + 159 eval sessions), all 6 classes present in both splits; **spsw** is very rare (78 total, 20 test). Own normalization stats computed (volts-scale, same lesson as Siena).

Experiment	Macro-F1	eyem F1	gped F1	pled F1	spsw F1
Frozen (no finetune)	0.302	0.468	0.430	0.265	0.054
Full finetune, no RL	0.278 (worse than frozen)	0.563	0.428	0.145	0.053
Full finetune + RL	0.321 (best)	0.639	0.479	0.189	0.047

Plain full-finetuning **overfit and underperformed frozen** (too little data: 1,796 train windows across 6 imbalanced classes for a 14.3M-param unfreeze). **Adding RL fixed the regression** and became the best TUEV result – RL’s macro-F1-shaped reward acted as useful regularization on this small task. **spsw** stays stuck near F1 0.05 regardless of method – a data-scarcity floor (20 test examples), not a training-strategy problem.

Cosmetic fix along the way: eval_processed_multiclass.py / finetune_processed_multiclass.py always printed “THUSZ global stats” as the normalization label regardless of which --norm-stats-dir was actually used (math was always correct; only the log line was misleading) – fixed to print the real path.

Results: Benchmarking/siena_binary/, siena_trainedhead_20260707_063623/, siena_finetuned_all_RL_20260707_10 tuev_trainedhead_20260707_072236/, tuev_finetuned_all_20260707_072601/, tuev_finetuned_all_RL_20260707_10

5.7 TUEV revisited — why was macro-F1 stuck at 0.32, and merging to 4 classes

6-class TUEV (5.6) plateaued around macro-F1 0.30-0.32 across frozen/finetune/+RL, and **spsw** never rose above F1 0.05-0.09. Diagnosed via confusion matrix: 83-97% of “spsw” *predictions* were actually gped or pled – the model wasn’t failing to find signal, it was systematically confusing spsw with its two closest relatives. Two candidate fixes were discussed:

- **(A) Soften class weighting.** The 6-class weighting used a hard inverse-frequency scheme (~8x ratio between rarest/commonest class); combined with RL’s macro-F1 reward, this was pushing the model to over-call the rare **spsw** class. Tried --weight-power 0.5 (sqrt-softened weights) + 100 epochs (up from 40, since the 40-epoch runs were still improving at the cutoff). **Result: fixed the mechanism** (spsw predictions dropped 192->88, closer to its true rate) **but not the outcome** – test macro-F1 barely moved (0.319 vs the original 0.321). The confusion just redistributed rather than resolved.
- **(B) Merge spsw+gped+pled into one epileptiform class.** These three are clinically the same discharge family, and (A) showed they’re genuinely inseparable at this embedding’s temporal resolution (5 s windows, coarse adaptive-avg-pooling likely destroys spike-level morphology that distinguishes them). Chosen after (A) confirmed the confusion wasn’t a fixable weighting artifact.

dataset_tools/merge_tuev_epileptiform.py relabels the **already-processed** 6-class data (spsw/gped/pled -> epileptiform; artf/bckg/eyem unchanged) – no EDF re-reading, reuses the same global normalization stats. Output: processed_TUEV_4class/. All obsolete 6-class scripts/results deleted first (eval_tuev.sh, eval_tuev_rl_v2.sh, old Benchmarking/tuev_* dirs) to keep the benchmarking set unambiguous.

Experiment	Accuracy	Macro-F1	epileptiform Precision	epileptiform Recall
Frozen (trained head only)	—	—	—	—
Full finetune + RL (weight-power 0.5, 100 epochs)	60.6%	0.499	90.9%	—

Merging to 4 classes was a large win: macro-F1 0.319 -> 0.499 (6-class RL run -> 4-class RL run), accuracy 33.9% -> 60.6%. Confirms the 6-class ceiling was a genuine resolution limit of the embedding, not a training-recipe problem – removing the indistinguishable-at-this-resolution distinction let the model actually use its capacity on the classes it *can* separate (artf/bckg/eyem vs. epileptiform-as-a-whole).

Results: Benchmarking/tuev_4class_trainedhead_20260708_071447/, tuev_4class_finetuned_all_RL_20260708_0720

5.8 sleep_edfx — a corpus-wide silent-corruption bug, found only by checking signal content

6-stage sleep staging (wake/n1/n2/n3/n4/rem), PhysioNet Sleep-EDF Expanded, 197 PSG+Hypnogram file pairs (5.5G raw). Three real bugs, found in sequence:

1. **PSG/Hypnogram pairing.** Exact-name `.replace("PSG.edf", "Hypnogram.edf")` never matched – the two filenames differ by a suffix character (e.g. SC4001E0-PSG.edf vs SC4001EC-Hypnogram.edf), not just the PSG/Hypnogram token. Fixed to glob by shared filename prefix.
2. **Label collapse.** `prepare_sleep_edfx` discarded the real sleep stage and reduced every annotation to binary wake/sleep even when `--multiclass` was requested. Fixed to preserve the actual 6-stage label.
3. **All-zero corpus (the big one).** Sleep-EDF’s PSG recordings carry only 2 EEG channels, and both are **bipolar derivations** (EEG Fpz-Cz, EEG Pz-Oz), not the monopolar/referential channels every other dataset (and the model’s target format) uses. Canonicalized to FPZCZ/PZ0Z, which matched **nothing** in the standard alias table – so all 22 target channels were zero-filled, for every window, across the **entire** 2.77M-window corpus, silently. This wasn’t caught by the initial validation pass (which only checked window counts/shapes) – it surfaced only after all 3 training runs collapsed to predicting a single constant class (“n4”) 100% of the time, at which point an explicit check of raw window statistics (`mean=0, std=0, 0/22 nonzero_channels` for every sample) confirmed the corruption. **Fixed** by adding FPZCZ->FZ / PZ0Z->PZ aliases (best-effort single-electrode approximation of each bipolar pair – not equivalent to a true monopolar channel, but the closest available mapping given the dataset’s real limitation). Wiped the corrupted processed data and all 3 tainted **Benchmarking/** results, reprocessed the full corpus (job 64888239, 42m38s), and re-verified this time with an **explicit non-zero-signal check** sampled across all 6 classes (confirmed exactly 2/22 channels carry real signal in every class – the maximum this dataset’s 2-EEG-channel limitation allows). Also fixed a **~35x performance bug** along the way (`resample_poly` was being called once per window instead of once per session) and a **memory leak** (`chunk = resampled[:, start:end]` was a numpy view keeping the whole parent session array alive in `BatchWriter`’s buffer across many files – fixed with an explicit `.copy()`).

Same 3-experiment suite as Siena/TUEV, on the reprocessed (verified-correct) data: 72,000 train / 72,000 test windows (`--max-per-class 12000, --per-window-norm` – this pipeline has no precomputed global stats), `--classes wake n1 n2 n3 n4 rem` (normal, a fallback bucket for unscored/gap epochs, dropped).

Mode	Accuracy	Macro-F1
Frozen (trained head only)	—	0.530
Full finetune (all, no RL)	53.5%	0.538
Full finetune + RL	53.9%	0.539

Per-class F1 (RL run): wake 0.83, n1 0.42, n2 0.35, n3 0.43, n4 0.68, rem 0.52 – consistent across both

finetuned variants. **wake** and **n4** (deep sleep) are well-separated; **n1/n2** (light-sleep transitions) are the weak point, which tracks known inter-rater difficulty in human sleep scoring, not obviously a model artifact.

Honest read: **RL gave essentially no lift here** (0.538 -> 0.539, within noise) – unlike Siena/TUEV where RL was the clear winner. Also notable: val macro-F1 climbed to 0.85 during training (per-window-normalized, class-balanced batches) while test macro-F1 landed at 0.54 – a real train/test distribution gap, not overfitting in the usual sense (no RL/no-RL difference), most likely driven by real-world class imbalance and difficulty in the untouched test split. The embedding carries real sleep-stage signal (54% vs 16.7% chance on 6 classes) despite this being the worst-case external dataset by channel coverage (2/22 real channels, both crude bipolar approximations) – but this is a harder, noisier transfer than Siena/TUEV, and RL’s usual regularization benefit didn’t materialize.

Results: `Benchmarking/sleep_edfx_trainedhead_20260708_084547/`, `sleep_edfx_finetuned_all_20260708_111752/`, `sleep_edfx_finetuned_all_RL_20260708_145647/`.

See also `CHANNEL_MAPPING_REPORT.md` for a full per-dataset breakdown of native channel counts and how each was mapped/zero-filled to the model’s 22-channel format, across all 7 datasets used in this project.

6. Key files

Scripts (finetuning/):

File	Purpose
<code>finetune_binary_PW.py</code>	THUSZ frozen +RL (original)
<code>finetune_binary_PW_unfreeze.py</code>	THUSZ +RL with selectable unfreeze levels
<code>finetune_binary_PW_norl.py</code>	THUSZ no-RL ablation (MLP head, weighted CE)
<code>eval_binary_PW.py</code>	Standalone eval of any binary checkpoint (ROC/PR/F1 + F1-optimal threshold)
<code>benchmark_prevalence.py</code>	Prevalence-matched re-scoring of saved probs (no GPU) for literature comparison
<code>finetune_tuab_PW.py</code>	TUAB progressive-unfreezing experiments (head_only..unfreeze_all)
<code>eval_processed_binary.py</code>	Zero-shot: reuse a pretrained binary classifier on any processed root, no training
<code>eval_processed_multiclass.py</code>	Frozen backbone + freshly trained head, N-class, any processed root (<code>--per-window-norm / --classes / --pooled-random-split</code>)
<code>finetune_processed_multiclass.py</code>	Backbone finetune (<code>--unfreeze none..all</code>), no RL, any processed root
<code>finetune_processed_rl.py</code>	Backbone finetune + Categorical-policy RL (macro-F1 reward), any processed root/class count
<code>*.sh</code>	matching SLURM launchers

Scripts (dataset_tools/):

File	Purpose
<code>download_public_eeg.py</code>	Download commands for external EEG datasets (Bonn/sleep_edfx/siena/mindbigdata/eegmmidb/bci_iv_2a)
<code>prepare_external_eeg.py</code>	EDF -> our 22ch/256Hz/1280-sample batch format; <code>--multiclass</code> keeps real labels
<code>convert_siena_npz.py</code>	Converts Siena’s pre-existing per-recording <code>.npz</code> into our standard batch format

File	Purpose
<code>prepare_tuev.py</code>	TUEV 6-class event corpus -> batches, event-driven windowing (see 5.6)
<code>merge_tuev_epileptiform.py</code>	Post-hoc relabel of processed TUEV data: <code>spsw+gped+pled</code> -> epileptiform (see 5.7)

Output folders (all inside EEGdiff_V2/):

Folder	Contents
<code>Binary_finetune_unfrozen/</code>	THUSZ +RL unfreeze runs (model + history + results); eval in <code>run_20260702_071240/</code>
<code>Binary_finetune_norl/</code>	THUSZ no-RL runs (frozen run in <code>run_norl_none_20260702_071546/</code>)
<code>TUHSZ_Res/</code>	THUSZ standalone eval results
<code>TUAB_Res/</code>	TUAB experiment runs
<code>Benchmarking/</code>	prevalence-matched lit. comparison + all external-dataset (Bonn/EEGMMIDB/Siena/TUEV) results
<code>PW_Seizure_Subtypes/</code>	subtype task: <code>extract_embeddings.py</code> , <code>cv_subtypes.py</code> , <code>finetune_cv_subtypes.py</code> , <code>cache/</code> , <code>cv_runs*/</code>
<code>logs/</code>	all SLURM <code>.out</code> / <code>.err</code>
<code>seizure_clf_20260629_084706/</code>	frozen +RL best checkpoint (in scratch root, original script)

External data (outside EEGdiff_V2/, under `Dataset/external_{raw,processed}/` and per-dataset dirs):

Path	Contents
<code>Dataset/external_processed/{bonn, eegmmidb, sleep_edfx}/</code>	processed batches for these 3 datasets
<code>Dataset/Siena/processed_siena/</code>	Siena's original npz-format processing (source for <code>convert_siena_npz.py</code>)
<code>Dataset/TUEV_v2.0.1/processed_TUEV/</code>	TUEV processed batches, 6-class (own <code>prepare_tuev.py</code>)
<code>Dataset/TUEV_v2.0.1/processed_TUEV_4class/</code>	TUEV relabeled to 4 classes (<code>merge_tuev_epileptiform.py</code> , see 5.7)
<code>Dataset/external_raw/sleep_edfx/</code>	<code>sleep_edfx</code> raw EDFs, 197/197 downloaded, processed (see 5.8)

7. Job history

Job IDs churn fast (dozens of short SLURM runs by now); the running log of `.out/.err` under `logs/` is the source of truth for any specific run. As of 2026-07-08, everything described in sections 5.1-5.8 above is **done**, including TUEV's 4-class merge and the full `sleep_edfx` pipeline (reprocessing job 64888239 + eval suite jobs 64889355/64899852 – the latter split in two because the combined 3-experiment job hit its 4h SLURM time limit mid-RL-run).

Note (2026-07-02): `finetune_tuab_PW.py` checkpoints `best.pth` on **every** val-F2 improvement (crash/timeout-safe), not just at experiment end.

8. Open items / next steps

- ☒ Process **sleep_edfx** (6-stage staging) and run the same trained-head/finetune/+RL suite used for Siena/TUEV (see 5.8).
- ☐ TUAB partial-unfreeze (**unfreeze_1**) was best on val but its run timed out before reaching test – get a real test number if still relevant.
- ☐ Address THUSZ unfrozen **overfitting** (stronger regularization, fewer epochs, or rely on frozen).
- ☐ For an “embedding model” paper: add a **linear probe** (single Linear on frozen 4800-d embedding, no RL), more downstream tasks, baselines (random-init, supervised-from-scratch, existing EEG foundation models), and ablations (which timesteps / layers matter).
- ☐ Cross-dataset pattern worth writing up explicitly: **RL helps most on small/imbalanced fine-tuning tasks** (fixed TUEV’s overfitting regression, sharply improved Siena’s precision) while **plain full-finetune (gentle LR, no RL) was already enough on larger/simpler tasks** (EEGMMIDB 2-class).
- ☐ Confirm patient-level disjointness of all splits explicitly.